PES UNIVERSITY

Department of Computer Science and Engineering

UE24CS343AB6: COMPUTER NETWORK SECURITY · ASSIGNMENT 2

Packet Sniffing & Spoofing using C

STUDENT

Sumedh Girish

INSTRUCTOR

Dr. Preet Kanwal

DATE

August 24, 2026

Setup

The lab setup relies on the Docker Compose environment provided in `Labsetup`, spawning three network entities: `attacker`, `HostA`, and `HostB`.

To start the containers, run:

```
docker compose up
```

Tasks

Before beginning with the lab, we observe that unlike in python, the C code requires us to interface direcctly with the raw bytes in the packet. Since calculating the offsets manually is error prone and hard to read, I define a C macro that allows wrapping the network headers in a way that allows access to inner elements using pointer arithmetic.

```
#define PACKET(Name, HeaderT) \
     typedef struct {            \
          HeaderT header;        \
          u_char data[];         \
     } Name

PACKET(eth_t, struct ether_header);
PACKET(ip_t, struct iphdr);
PACKET(icmp_t, struct icmphdr);
PACKET(tcp_t, struct tcphdr);
```

> **Question 1 · Understanding how a Sniffer Works**
> In this task, students need to write a sniffer program to print out the source and destination IP addresses of each captured packet.

SOLUTION

Before we can access any packets, we must first connect to the device NIC adapter through our code. We do this using the `pcap_findalldevs` function that returns a pointer to a linked list containing all available NIC adapters on the system.

Below we demonstrate using the first device to sniff the network.

```
pcap_if_t* alldevsp = NULL;
char errbuf[PCAP_ERRBUF_SIZE];

pcap_findalldevs(&alldevsp, errbuf);
char* default_dev = alldevsp[0].name;

pcap_t* handle = pcap_open_live(default_dev, 262144, 1, 1000, errbuf);
```

> **NOTE**
> We require priviledged mode to run this program. This is because the code to access the NIC configures it to use promiscus mode, which is a high priviledged operation. Instead doing
>
> ```
> pcap_t* handle = pcap_open_live(default_dev, 262144, 0, 1000, errbuf);
> ```
>
> disables promiscus mode and external packets become invisible to our process - and the application can be then run without root.

BPF Filter Compilation

Packet filtering is implemented at the kernel level using Berkeley Packet Filters (BPF). We compile the filter expression and bind it to the capture handle. This allows us to apply filters at the kernel level into driver hardware - which makes filtering extremely efficient.

```
void apply_filter(pcap_t* handle, char* filter_exp) {
    struct bpf_program fp;
    bpf_u_int32 net;

    pcap_compile(handle, &fp, filter_exp, 1, net);
    LOG_ON_ERROR(pcap_setfilter(handle, &fp), handle);
}
```

As required, we apply the following example filters to the listener.

```
apply_filter(handle, "tcp src portrange 10-100");
apply_filter(handle, "tcp dst port 21")
```

Packet Dissection Callback

The `pcap_loop` function invokes `print_packet` upon frame arrival. We step down the network stack from Ethernet (`eth_t`), through IP (`ip_t`), down to TCP (`tcp_t`):

Since all data transmitted is a payload to TCP, **any passwords transmitted should also be visible via this log**.

```
void print_packet(u_char* user, const struct pcap_pkthdr* header, const u_char*
packet) {
    eth_t* link_pkt = (eth_t*)packet;

    if (ntohs(link_pkt->header.ether_type) == ETHERTYPE_IP) {
        ip_t* ip_pkt = (ip_t*)link_pkt->data;

        char src_ip[INET_ADDRSTRLEN], dst_ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &ip_pkt->header.saddr, src_ip, INET_ADDRSTRLEN);
        inet_ntop(AF_INET, &ip_pkt->header.daddr, dst_ip, INET_ADDRSTRLEN);
        struct protoent* proto = getprotobynumber(ip_pkt->header.protocol);

        printf("IP(src_ip: %s, dst_ip: %s, proto: %s)", src_ip, dst_ip, proto-
>p_name);

        if (ip_pkt->header.protocol == IPPROTO_TCP) {
            tcp_t* tcp_pkt = (tcp_t*)ip_pkt->data;
            printf(" / TCP(src_port: %d, dst_port: %d) / Data(%s)",
                   tcp_pkt->header.th_sport, tcp_pkt->header.th_dport,
tcp_pkt->data);
        }
        printf("\n");
    }
}
```

Question 2 · Packet Spoofing with Raw Sockets

The objective of this task is to craft custom raw IP packets programmatically and transmit them using raw sockets in C.

SOLUTION

Internet Checksum Calculation

IP and ICMP protocols require 16-bit one's complement checksums across their respective headers. The code for this in the reference implementation is **wrong** and can overflow. The fixed code is given below.

```
unsigned short in_checksum(unsigned short* data, int length) {
     unsigned short* curr = data;
     unsigned int sum = 0;

     while (length > 1) {
          sum += *curr++;
          length -= 2;
     }

     if (length == 1) {
          unsigned short temp = 0;
          *(u_char*)&temp = *(u_char*)curr;
          sum += temp;
     }

     while (sum >> 16) {
          sum = (sum & 0xffff) + (sum >> 16);
     }

     return (unsigned short)(~sum);
}
```

Raw Socket Transmission (`IP_HDRINCL`)

To inject custom IP packets with spoofed header parameters, we open an `AF_INET` raw socket and enable `IP_HDRINCL` to inform the Linux kernel that the IP header is provided directly by our program.

```
void send_icmp_packet(ip_t* pkt, int total_len) {
     int sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
     if (sock < 0) {
          perror("socket error");
          return;
     }

     int header_incl = 1;
     if (setsockopt(sock, IPPROTO_IP, IP_HDRINCL, &header_incl,
sizeof(header_incl)) < 0) {
          perror("setsockopt error");
          close(sock);
```

```
            return;
        }

        struct sockaddr_in dest;
        memset(&dest, 0, sizeof(dest));
        dest.sin_family = AF_INET;
        memcpy(&dest.sin_addr, &pkt->header.daddr, sizeof(pkt->header.daddr));

        if (sendto(sock, pkt, total_len, 0, (struct sockaddr*)&dest, sizeof(dest))
< 0) {
            perror("sendto error");
        }

        close(sock);
}
```

Since we already have a constructed packet ready in the `print_packet` function, we modify those fields beofre sending the required packet back as required.

Question 3 · Sniffing and-then Spoofing in C
Implement an automated sniffer-spoofer daemon that intercepts incoming ICMP Echo Requests on the local interface and immediately synthesizes a spoofed ICMP Echo Reply back to the sender.

SOLUTION

Spoofing Logic Flow

The callback function intercepts incoming frames, filters for ICMP Echo Requests (`type == 8`), swaps network layer source/destination addressing, adjusts ICMP message types to Echo Reply (`type == 0`), recomputes checksums, and dispatches the frame.

```
void spoofer(u_char* user, const struct pcap_pkthdr* header, const u_char*
packet) {
     int ip_hdr_len = sizeof(struct iphdr);
     int min_len = sizeof(eth_t) + ip_hdr_len + sizeof(struct icmphdr);

     if ((int)header->caplen < min_len) return;

     eth_t* eth_pkt = (eth_t*)packet;

     if (ntohs(eth_pkt->header.ether_type) == ETHERTYPE_IP) {
          ip_t* ip_pkt = (ip_t*)eth_pkt->data;

          if (ip_pkt->header.protocol == IPPROTO_ICMP) {
               icmp_t* icmp_pkt = (icmp_t*)ip_pkt->data;

               // Handle ICMP Echo Requests (Type 8)
               if (icmp_pkt->header.type == ICMP_ECHO) {
                    int total_ip_len = ntohs(ip_pkt->header.tot_len);
                    int icmp_len = total_ip_len - (ip_pkt->header.ihl * 4);

                    u_char* reply_buf = malloc(total_ip_len);
                    if (!reply_buf) return;

                    memcpy(reply_buf, ip_pkt, total_ip_len);
                    ip_t* reply_ip = (ip_t*)reply_buf;
                    icmp_t* reply_icmp = (icmp_t*)reply_ip->data;

                    // 1. Convert Echo Request to Echo Reply
                    reply_icmp->header.type = ICMP_ECHOREPLY;
                    reply_icmp->header.checksum = 0;
                    reply_icmp->header.checksum = in_checksum((unsigned
short*)reply_icmp, icmp_len);

                    // 2. Swap Source and Destination IP Addresses
                    swap(&reply_ip->header.saddr, &reply_ip->header.daddr);

                    // 3. Recalculate IP Header Checksum
                    reply_ip->header.check = 0;
                    reply_ip->header.check = in_checksum((unsigned
```

```
short*)reply_ip, reply_ip->header.ihl * 4);

                    // 4. Send the spoofed reply
                    send_icmp_packet(reply_ip, total_ip_len);

                    free(reply_buf);
                }
            }
        }
}
```

Main Loop Execution

```
int main(int argc, const char* argv[]) {
    pcap_if_t* alldevsp = NULL;
    char errbuf[PCAP_ERRBUF_SIZE];

    if (pcap_findalldevs(&alldevsp, errbuf) < 0 || alldevsp == NULL) return
EXIT_FAILURE;

    pcap_t* handle = pcap_open_live(alldevsp->name, 262144, 1, 1000, errbuf);
    apply_filter(handle, "icmp and icmp[icmptype] == icmp-echo");

    printf("Sniffing for ICMP Echo Requests...\n");
    pcap_loop(handle, -1, spoofer, NULL);

    pcap_close(handle);
    pcap_freealldevs(alldevsp);
    return EXIT_SUCCESS;
}
```

C Code Summary

TCP Sniffer Implementation (`sniffer.c`)

```c
#include <assert.h>
#include <pcap/pcap.h>
#include <stdio.h>
#include <stdlib.h>

#include <arpa/inet.h>
#include <net/ethernet.h>
#include <netdb.h>
#include <netinet/ether.h>
#include <netinet/if_ether.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>

#define PACKET(Name, HeaderT)    \
     typedef struct {              \
          HeaderT header;          \
          u_char data[];           \
     } Name

PACKET(eth_t, struct ether_header);
PACKET(ip_t, struct iphdr);
PACKET(tcp_t, struct tcphdr);

void print_packet(u_char* user, const struct pcap_pkthdr* header, const u_char*
packet) {
     eth_t* link_pkt = (eth_t*)packet;

     if (ntohs(link_pkt->header.ether_type) == ETHERTYPE_IP) {
          ip_t* ip_pkt = (ip_t*)link_pkt->data;

          char src_ip[INET_ADDRSTRLEN];
          char dst_ip[INET_ADDRSTRLEN];

          inet_ntop(AF_INET, &ip_pkt->header.saddr, src_ip, INET_ADDRSTRLEN);
          inet_ntop(AF_INET, &ip_pkt->header.daddr, dst_ip, INET_ADDRSTRLEN);
          struct protoent* proto = getprotobynumber(ip_pkt->header.protocol);

          printf("IP(");
          printf("src_ip: %s, ", src_ip);
          printf("dst_ip: %s, ", dst_ip);
          printf("proto: %s", proto->p_name);
          printf(")");

          if (ip_pkt->header.protocol == IPPROTO_TCP) {
               tcp_t* tcp_pkt = (tcp_t*)ip_pkt->data;

               printf(" / TCP(");
               printf("src_port: %d, ", tcp_pkt->header.th_sport);
```

```
                printf("dst_port: %d", tcp_pkt->header.th_dport);
                printf(")");
                printf(" / Data(%s)", tcp_pkt->data);
           }

           printf("\n");
      }
}

#define LOG_ON_ERROR(exp, handle)          \
     if (exp == 0) {                           \
          pcap_perror(handle, "Error: ");   \
          exit(EXIT_FAILURE);                 \
     }

void apply_filter(pcap_t* handle, char* filter_exp) {
     struct bpf_program fp;
     bpf_u_int32 net;

     pcap_compile(handle, &fp, filter_exp, 1, net);
     LOG_ON_ERROR(pcap_setfilter(handle, &fp), handle);
}

int main(int argc, const char* argv[]) {
     pcap_if_t* alldevsp = NULL;
     char errbuf[PCAP_ERRBUF_SIZE];

     pcap_findalldevs(&alldevsp, errbuf);

     char* default_dev = alldevsp[0].name;
     printf("Using device %s.\n", default_dev);

     pcap_t* handle = pcap_open_live(default_dev, 262144, 1, 1000, errbuf);

     apply_filter(handle, "tcp port 80");

     pcap_loop(handle, -1, print_packet, NULL);

     pcap_close(handle);
}
```

ICMP Sniffer & Spoofer Implementation (`spoofer.c`)

```c
#include <assert.h>
#include <pcap/pcap.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include <arpa/inet.h>
#include <net/ethernet.h>
#include <netdb.h>
#include <netinet/ether.h>
#include <netinet/if_ether.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/ip_icmp.h>
#include <sys/socket.h>

#define PACKET(Name, HeaderT)  \
     typedef struct {             \
          HeaderT header;         \
          u_char data[];          \
     } Name

PACKET(eth_t, struct ether_header);
PACKET(ip_t, struct iphdr);
PACKET(icmp_t, struct icmphdr);

void swap(unsigned int* a, unsigned int* b) {
     unsigned int tmp = *b;
     *b = *a;
     *a = tmp;
}

unsigned short in_checksum(unsigned short* data, int length) {
     unsigned short* curr = data;
     unsigned int sum = 0;

     while (length > 1) {
          sum += *curr++;
          length -= 2;
     }

     if (length == 1) {
          unsigned short temp = 0;
          *(u_char*)&temp = *(u_char*)curr;
          sum += temp;
     }

     while (sum >> 16) {
          sum = (sum & 0xffff) + (sum >> 16);
     }

     return (unsigned short)(~sum);
```

```
}

void send_icmp_packet(ip_t* pkt, int total_len) {
     int sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
     if (sock < 0) {
          perror("socket error");
          return;
     }

     int header_incl = 1;
     if (setsockopt(sock, IPPROTO_IP, IP_HDRINCL, &header_incl, sizeof(header_incl))
< 0) {
          perror("setsockopt error");
          close(sock);
          return;
     }

     struct sockaddr_in dest;
     memset(&dest, 0, sizeof(dest));
     dest.sin_family = AF_INET;
     memcpy(&dest.sin_addr, &pkt->header.daddr, sizeof(pkt->header.daddr));

     if (sendto(sock, pkt, total_len, 0, (struct sockaddr*)&dest, sizeof(dest)) < 0)
{
          perror("sendto error");
     }

     close(sock);
}

void spoofer(u_char* user, const struct pcap_pkthdr* header, const u_char* packet)
{
     int ip_hdr_len = sizeof(struct iphdr);
     int min_len = sizeof(eth_t) + ip_hdr_len + sizeof(struct icmphdr);

     if ((int)header->caplen < min_len)
          return;

     eth_t* eth_pkt = (eth_t*)packet;

     if (ntohs(eth_pkt->header.ether_type) == ETHERTYPE_IP) {
          ip_t* ip_pkt = (ip_t*)eth_pkt->data;

          if (ip_pkt->header.protocol == IPPROTO_ICMP) {
               icmp_t* icmp_pkt = (icmp_t*)ip_pkt->data;

               // Handle ICMP Echo Requests (Type 8)
               if (icmp_pkt->header.type == ICMP_ECHO) {
                    int total_ip_len = ntohs(ip_pkt->header.tot_len);
                    int icmp_len = total_ip_len - (ip_pkt->header.ihl * 4);

                    // Create a mutable copy of the IP frame
                    u_char* reply_buf = malloc(total_ip_len);
                    if (!reply_buf)
```

```
                    return;

                memcpy(reply_buf, ip_pkt, total_ip_len);
                ip_t* reply_ip = (ip_t*)reply_buf;
                icmp_t* reply_icmp = (icmp_t*)reply_ip->data;

                // 1. Convert Echo Request to Echo Reply
                reply_icmp->header.type = ICMP_ECHOREPLY;
                reply_icmp->header.checksum = 0;
                reply_icmp->header.checksum = in_checksum((unsigned
short*)reply_icmp, icmp_len);

                // 2. Swap Source and Destination IP Addresses
                swap(&reply_ip->header.saddr, &reply_ip->header.daddr);

                // 3. Recalculate IP Header Checksum
                reply_ip->header.check = 0;
                reply_ip->header.check =
                    in_checksum((unsigned short*)reply_ip, reply_ip->header.ihl *
4);

                // 4. Send the spoofed reply
                send_icmp_packet(reply_ip, total_ip_len);

                free(reply_buf);
            }
        }
    }
}

#define LOG_ON_ERROR(exp, handle)          \
    if (exp != 0) {                            \
        pcap_perror(handle, "Error: ");   \
        exit(EXIT_FAILURE);                  \
    }

void apply_filter(pcap_t* handle, const char* filter_exp) {
    struct bpf_program fp;
    bpf_u_int32 net = PCAP_NETMASK_UNKNOWN;

    if (pcap_compile(handle, &fp, filter_exp, 1, net) < 0) {
        pcap_perror(handle, "pcap_compile error");
        exit(EXIT_FAILURE);
    }
    LOG_ON_ERROR(pcap_setfilter(handle, &fp), handle);
    pcap_freecode(&fp);
}

int main(int argc, const char* argv[]) {
    pcap_if_t* alldevsp = NULL;
    char errbuf[PCAP_ERRBUF_SIZE];

    if (pcap_findalldevs(&alldevsp, errbuf) < 0 || alldevsp == NULL) {
        fprintf(stderr, "Error finding devices: %s\n", errbuf);
```

```
        return EXIT_FAILURE;
    }

    char* default_dev = alldevsp->name;
    printf("Using device %s.\n", default_dev);

    pcap_t* handle = pcap_open_live(default_dev, 262144, 1, 1000, errbuf);
    if (!handle) {
        fprintf(stderr, "pcap_open_live failed: %s\n", errbuf);
        pcap_freealldevs(alldevsp);
        return EXIT_FAILURE;
    }

    apply_filter(handle, "icmp and icmp[icmptype] == icmp-echo");

    printf("Sniffing for ICMP Echo Requests...\n");
    pcap_loop(handle, -1, spoofer, NULL);

    pcap_close(handle);
    pcap_freealldevs(alldevsp);
    return EXIT_SUCCESS;
}
```

Results

Question 4 · Promiscuous Mode

Demonstrate the difference when promiscuous mode is turned on (`1`) versus turned off (`0`).

SOLUTION

Promiscuous mode alters host network card filtering behavior at hardware layer:

- Promiscuous Mode Enabled (`1`) : The NIC passes all captured Ethernet frames on the collision domain to the host stack, regardless of destination MAC address.
- Promiscuous Mode Disabled (`0`): The NIC hardware filters out frames whose destination MAC address does not match the attacker host's physical MAC or broadcast address.

Observed Execution Behavior: When Host A (`10.9.0.5`) pings Host B (`10.9.0.6`):

- Mode `1` : Attacker sniffer logs ICMP packet flow between `10.9.0.5` and `10.9.0.6` successfully.
- Mode `0` : Non-multicast/broadcast traffic between external hosts is silently discarded in hardware; the sniffer logs zero frames.

Question 5 · Question 4: IP Header Checksum in Raw Sockets

Using raw socket programming, do you have to calculate the checksum for the IP header?

SOLUTION

When creating a socket using `SOCK_RAW` with `IP_HDRINCL` enabled, the programmer provides a custom IP header. Although modern Linux kernels recompute and overwrite zeroed IP checksums (`ip→check = 0`) automatically upon transmission via raw sockets, it is good practice and more reliable to do it ourselves.

Question 6 · Question 5: Privilege Requirements for Raw Sockets

Why do you need root privilege to run programs that use raw sockets? Where does the program fail if executed without root?

SOLUTION

Raw sockets bypass normal transport layer socket wrappers, allowing arbitrary source IP address forging and raw packet crafting. To prevent unauthorized IP spoofing and denial-of-service attacks, Linux restricts `SOCK_RAW` creation to processes possessing `CAP_NET_RAW` capability.

When executed as an unprivileged user, the program fails immediately at socket creation:

```
$ ./spooficmp
socket error: Operation not permitted
```

The call `socket(AF_INET, SOCK_RAW, IPPROTO_RAW)` returns `-1` (`EPERM`).

Task 2.1 A

Executing `sniff` on the attacker host (`10.9.0.1`) while Host A (`10.9.0.5`) pings the gateway logs packet flows correctly:

```
Using device eth0.
IP(src_ip: 10.9.0.5, dst_ip: 10.9.0.1, proto: icmp)
```

Task 2.1 C (Password Extraction)

Telnet transmits data in plain unencrypted ASCII. By reading `tcp_pkt→data`, typed characters in authentication streams appear directly in captured payload output.

Executing `spooficmp` sends forged ICMP Echo Requests with a spoofed source address (`10.9.0.5`) toward target `8.8.8.8`:

Host A (`10.9.0.5`) executes `ping 1.2.3.4` for a non-existent host address on the network:

1. Attacker daemon captures `ICMP Echo Request` (`Type 8`) targeting `1.2.3.4`.
2. Attacker synthesizes an `ICMP Echo Reply` (`Type 0`) setting source IP to `1.2.3.4` and destination IP to `10.9.0.5`, recomputes ICMP/IP checksums, and dispatches frame via raw socket.
3. Host A terminal receives immediate reply:

```
PING 1.2.3.4 (1.2.3.4) 56(84) bytes of data.
64 bytes from 1.2.3.4: icmp_seq=1 ttl=64 time=0.412 ms
64 bytes from 1.2.3.4: icmp_seq=2 ttl=64 time=0.389 ms
...
```

Because docker containers dont have pcap library to dynamically link to and my host machine on linux doesnt connect to ethernet, demonstrating the telnet connection over ethernet specifically for the given exercieses is impossible. However I have executed equivatents for monitoring http traffic and checked that the output is sane.